

Base code:

```
#include <stdint.h>

// -----
// REGISTER BASE ADDRESSES
// -----
#define RCC_BASE    0x40023800
#define GPIOA_BASE  0x40020000
#define GPIOB_BASE  0x40020400
#define GPIOD_BASE  0x40020C00
#define ADC1_BASE   0x40012000
#define ADC_COMMON  0x40012300
#define I2C1_BASE   0x40005400
#define USART2_BASE 0x40004400

// RCC
#define RCC_AHB1ENR (*(volatile uint32_t*)(RCC_BASE + 0x30))
#define RCC_APB1ENR (*(volatile uint32_t*)(RCC_BASE + 0x40))
#define RCC_APB2ENR (*(volatile uint32_t*)(RCC_BASE + 0x44))

// GPIOA
#define GPIOA_MODER (*(volatile uint32_t*)(GPIOA_BASE + 0x00))
#define GPIOA_AFR_L (*(volatile uint32_t*)(GPIOA_BASE + 0x20))
#define GPIOA_ODR   (*(volatile uint32_t*)(GPIOA_BASE + 0x14))

// GPIOB
#define GPIOB_MODER (*(volatile uint32_t*)(GPIOB_BASE + 0x00))
#define GPIOB_AFR_L (*(volatile uint32_t*)(GPIOB_BASE + 0x20))

// GPIOD
#define GPIOD_MODER (*(volatile uint32_t*)(GPIOD_BASE + 0x00))
#define GPIOD_ODR   (*(volatile uint32_t*)(GPIOD_BASE + 0x14))

// ADC1
#define ADC1_SR    (*(volatile uint32_t*)(ADC1_BASE + 0x00))
#define ADC1_CR1   (*(volatile uint32_t*)(ADC1_BASE + 0x04))
#define ADC1_CR2   (*(volatile uint32_t*)(ADC1_BASE + 0x08))
#define ADC1_SMPR2 (*(volatile uint32_t*)(ADC1_BASE + 0x14))
#define ADC1_SQR1  (*(volatile uint32_t*)(ADC1_BASE + 0x2C))
#define ADC1_SQR3  (*(volatile uint32_t*)(ADC1_BASE + 0x34))
#define ADC1_DR    (*(volatile uint32_t*)(ADC1_BASE + 0x4C))
#define ADC_CCR    (*(volatile uint32_t*)(ADC_COMMON + 0x04))

// I2C1
#define I2C1_CR1   (*(volatile uint32_t*)(I2C1_BASE + 0x00))
#define I2C1_CR2   (*(volatile uint32_t*)(I2C1_BASE + 0x04))
#define I2C1_DR    (*(volatile uint32_t*)(I2C1_BASE + 0x10))
#define I2C1_SR1   (*(volatile uint32_t*)(I2C1_BASE + 0x14))
#define I2C1_SR2   (*(volatile uint32_t*)(I2C1_BASE + 0x18))
```

Base code:

```
#define I2C1_CCR    (*(volatile uint32_t*)(I2C1_BASE + 0x1C))
#define I2C1_TRISE  (*(volatile uint32_t*)(I2C1_BASE + 0x20))

// USART2
#define USART2_SR    (*(volatile uint32_t*)(USART2_BASE + 0x00))
#define USART2_DR    (*(volatile uint32_t*)(USART2_BASE + 0x04))
#define USART2_BRR    (*(volatile uint32_t*)(USART2_BASE + 0x08))
#define USART2_CR1    (*(volatile uint32_t*)(USART2_BASE + 0x0C))

// GPIOA ODR bits for fan and motor control
#define FAN_PIN    8 // PA8 - fan transistor
#define MOTOR_PIN  9 // PA9 - motor transistor

// -----
// DELAY
// -----
void delay_ms(volatile uint32_t ms) {
    for (; ms > 0; ms--)
        for (volatile uint32_t i = 0; i < 3360; i++);
}

// -----
// USART2 — PA2 TX, 9600 baud @ 16MHz HSI
// -----
void usart2_init(void) {
    RCC_AHB1ENR |= (1 << 0);
    RCC_APB1ENR |= (1 << 17);

    // PA2 → AF mode
    GPIOA_MODER &= ~(3 << 4);
    GPIOA_MODER |= (2 << 4);
    GPIOA_AFRL &= ~(0xF << 8);
    GPIOA_AFRL |= (7 << 8); // AF7 = USART2

    USART2_BRR = 0x0683; // 9600 baud @ 16MHz
    USART2_CR1 |= (1 << 13)|(1 << 3);
}

void usart2_send_char(char c) {
    while (!(USART2_SR & (1 << 7)));
    USART2_DR = c;
}

void usart2_print(const char *str) {
    while (*str) usart2_send_char(*str++);
}

void print_number(int32_t val) {
```

Base code:

```
char buf[12];
int i = 11;
buf[i] = '\0';
if (val == 0) { usart2_send_char('0'); return; }
uint8_t neg = 0;
if (val < 0) { neg = 1; val = -val; }
while (val > 0) { buf[--i] = '0' + (val % 10); val /= 10; }
if (neg) buf[--i] = '-';
usart2_print(&buf[i]);
}

void lcd_print_number(int32_t val); // forward declaration

// -----
// ADC1 — PA0 (CH0 current), PA1 (CH1 voltage), PA3 (CH3 temp)
// -----
void adc1_init(void) {
    RCC_AHB1ENR |= (1 << 0);
    RCC_APB2ENR |= (1 << 8);

    // PA0, PA1, PA3 → Analog mode
    GPIOA_MODER |= (3<<0)|(3<<2)|(3<<6);

    ADC_CCR &= ~(3 << 16);
    ADC_CCR |= (1 << 16); // prescaler /4

    // Sample time CH0, CH1, CH3
    ADC1_SMPR2 |= (7<<0)|(7<<3)|(7<<9);

    ADC1_CR2 |= (1 << 0); // ADON
    delay_ms(1);
}

uint32_t adc_read(uint8_t channel) {
    ADC1_SQR3 = channel;
    ADC1_SQR1 &= ~(0xF << 20);
    ADC1_CR2 |= (1 << 30); // SWSTART
    while (!(ADC1_SR & (1 << 1)));
    return ADC1_DR;
}

// -----
// I2C1 — PB6 SCL, PB7 SDA
// -----
void i2c1_init(void) {
    RCC_AHB1ENR |= (1 << 1);
    RCC_APB1ENR |= (1 << 21);
```

Base code:

```
GPIOB_MODER &= ~((3<<12)|(3<<14));
GPIOB_MODER |= ((2<<12)|(2<<14));

volatile uint32_t *GPIOB_AFRH = (volatile uint32_t*)(GPIOB_BASE + 0x24);
volatile uint32_t *GPIOB_OTYPER= (volatile uint32_t*)(GPIOB_BASE + 0x04);
volatile uint32_t *GPIOB_PUPDR = (volatile uint32_t*)(GPIOB_BASE + 0x0C);

*GPIOB_AFRH &= ~((0xF<<24)|(0xF<<28));
*GPIOB_AFRH |= ((4<<24)|(4<<28)); // AF4
*GPIOB_OTYPER|= (1<<6)|(1<<7); // open drain
*GPIOB_PUPDR &= ~((3<<12)|(3<<14));
*GPIOB_PUPDR |= ((1<<12)|(1<<14)); // pull up

I2C1_CR2 = 16;
I2C1_CCR = 80;
I2C1_TRISE = 17;
I2C1_CR1 |= (1 << 0);
}

void i2c_start(void) {
    I2C1_CR1 |= (1<<8);
    while (!(I2C1_SR1 & (1<<0)));
}

void i2c_write_addr(uint8_t addr) {
    I2C1_DR = addr;
    while (!(I2C1_SR1 & (1<<1)));
    (void)I2C1_SR2;
}

void i2c_write_data(uint8_t data) {
    while (!(I2C1_SR1 & (1<<7)));
    I2C1_DR = data;
}

void i2c_stop(void) {
    while (!(I2C1_SR1 & (1<<2)));
    I2C1_CR1 |= (1<<9);
}

void i2c_send_byte(uint8_t addr, uint8_t data) {
    i2c_start();
    i2c_write_addr(addr);
    i2c_write_data(data);
    i2c_stop();
}

// -----
```

Base code:

```
// LCD DRIVER
// -----
#define LCD_ADDR    (0x27 << 1)
#define LCD_BACKLIGHT 0x08
#define LCD_EN      0x04
#define LCD_RS      0x01

void lcd_send_nibble(uint8_t nibble, uint8_t mode) {
    uint8_t data = nibble | LCD_BACKLIGHT | mode;
    i2c_send_byte(LCD_ADDR, data);
    i2c_send_byte(LCD_ADDR, data | LCD_EN);
    delay_ms(1);
    i2c_send_byte(LCD_ADDR, data & ~LCD_EN);
}

void lcd_send_byte_fn(uint8_t byte, uint8_t mode) {
    lcd_send_nibble(byte & 0xF0, mode);
    lcd_send_nibble((byte << 4) & 0xF0, mode);
}

void lcd_cmd(uint8_t cmd) { lcd_send_byte_fn(cmd, 0x00); }
void lcd_char(uint8_t chr) { lcd_send_byte_fn(chr, LCD_RS); }

void lcd_init(void) {
    delay_ms(50);
    lcd_send_nibble(0x30, 0); delay_ms(5);
    lcd_send_nibble(0x30, 0); delay_ms(1);
    lcd_send_nibble(0x30, 0); delay_ms(1);
    lcd_send_nibble(0x20, 0);
    lcd_cmd(0x28);
    lcd_cmd(0x0C);
    lcd_cmd(0x06);
    lcd_cmd(0x01);
    delay_ms(2);
}

void lcd_set_cursor(uint8_t row, uint8_t col) {
    lcd_cmd((row == 0) ? (0x80 + col) : (0xC0 + col));
}

void lcd_print(const char *str) {
    while (*str) lcd_char((uint8_t)*str++);
}

void lcd_clear(void) { lcd_cmd(0x01); delay_ms(2); }

void lcd_print_number(int32_t val) {
    char buff[12];
```

Base code:

```
int i = 11;
buf[i] = '\0';
if (val == 0) { lcd_char('0'); return; }
uint8_t neg = 0;
if (val < 0) { neg = 1; val = -val; }
while (val > 0) { buf[--i] = '0' + (val % 10); val /= 10; }
if (neg) buf[--i] = '-';
lcd_print(&buf[i]);
}

// -----
// GPIO — Fan (PA8) and Motor (PA9) as output
// -----
void gpio_output_init(void) {
    RCC_AHB1ENR |= (1 << 0); // GPIOA

    // PA8, PA9 → Output mode (01)
    GPIOA_MODER &= ~((3<<16)|(3<<18));
    GPIOA_MODER |= ((1<<16)|(1<<18));
}

void fan_on(void) { GPIOA_ODR |= (1 << FAN_PIN); }
void fan_off(void) { GPIOA_ODR &= ~(1 << FAN_PIN); }
void motor_on(void) { GPIOA_ODR |= (1 << MOTOR_PIN); }
void motor_off(void) { GPIOA_ODR &= ~(1 << MOTOR_PIN); }

// -----
// SENSOR CONVERSIONS
// -----
int32_t get_current(uint32_t raw) {
    // ACS712-20A: 100mV/A, midpoint 2048
    int32_t mv = ((int32_t)raw - 2048) * 3300 / 4095;
    int32_t i = mv / 10; // in 0.1A
    if (i > -1 && i < 1) i = 0;
    return i;
}

int32_t get_voltage(uint32_t raw) {
    // Divider: 56k + 10k → factor 6.6
    int32_t vadc_mv = (int32_t)raw * 3300 / 4095;
    int32_t vbat_mv = vadc_mv * 66 / 10;
    return vbat_mv / 100; // in 0.1V
}

int32_t get_temperature(uint32_t raw) {
    // LM35: 10mV/°C, 3.3V ref
    int32_t mv = (int32_t)raw * 3300 / 4095;
    return mv / 10; // °C
}
```

Base code:

```
}

// -----
// SOC — COULOMB COUNTING
// -----
int32_t charge_mAs = 15480000; // 4300mAh fully charged
int32_t capacity_mAs = 15480000;

void update_soc(int32_t current_10, uint32_t dt_ms) {
    int32_t delta = (current_10 * 100 * (int32_t)dt_ms) / 1000;
    charge_mAs -= delta;
    if (charge_mAs < 0) charge_mAs = 0;
    if (charge_mAs > capacity_mAs) charge_mAs = capacity_mAs;
}

int32_t get_soc(void) {
    return (charge_mAs * 100) / capacity_mAs;
}

// -----
// PROTECTION ALERTS
// -----
// 2S Li-ion: max 8.4V (84 in 0.1V), min 6.0V (60 in 0.1V)
// Overcurrent: 3A (30 in 0.1A)
// Overtemp: 45°C

void check_protection(int32_t voltage, int32_t current,
                     int32_t temp, int32_t soc) {
    if (voltage > 84) {
        lcd_clear();
        lcd_set_cursor(0, 0); lcd_print("!! OVERVOLTAGE !!");
        lcd_set_cursor(1, 0); lcd_print("Disconnect Pack ");
        usart2_print("FAULT:OVERVOLTAGE\r\n");
        delay_ms(2000);
    }
    else if (voltage < 60 && voltage > 10) {
        lcd_clear();
        lcd_set_cursor(0, 0); lcd_print("!! UNDERVOLTAGE!");
        lcd_set_cursor(1, 0); lcd_print("Charge Battery ");
        usart2_print("FAULT:UNDERVOLTAGE\r\n");
        delay_ms(2000);
    }
    else if (current > 30) {
        lcd_clear();
        lcd_set_cursor(0, 0); lcd_print("!! OVERCURRENT !");
        lcd_set_cursor(1, 0); lcd_print("Check Load ");
        usart2_print("FAULT:OVERCURRENT\r\n");
        delay_ms(2000);
    }
}
```

Base code:

```
}
else if (temp > 45) {
    lcd_clear();
    lcd_set_cursor(0, 0); lcd_print("!! OVERTEMP !! ");
    lcd_set_cursor(1, 0); lcd_print("Cooling Active ");
    usart2_print("FAULT:OVERTEMP\r\n");
    delay_ms(2000);
}
// Low SoC warning
if (soc < 20) {
    usart2_print("WARNING:LOW_SOC\r\n");
}
}

// -----
// MAIN
// -----
int main(void) {
    RCC_AHB1ENR |= (1<<0)|(1<<1)|(1<<3);

    usart2_init();
    adc1_init();
    i2c1_init();
    gpio_output_init();
    lcd_init();

    lcd_clear();
    lcd_set_cursor(0, 0); lcd_print(" EVBMS v1.0 ");
    lcd_set_cursor(1, 0); lcd_print(" Initializing.. ");
    delay_ms(2000);
    lcd_clear();

    usart2_print("V(0.1V),I(0.1A),T(C),SoC(%),Fan,Motor\r\n");

    uint32_t ms = 0;
    uint32_t last_ms = 0;

    while (1) {
        delay_ms(500);
        ms += 500;
        uint32_t dt_ms = ms - last_ms;
        last_ms = ms;

        // Read sensors
        uint32_t raw_i = adc_read(0); // PA0 current
        uint32_t raw_v = adc_read(1); // PA1 voltage
        uint32_t raw_t = adc_read(3); // PA3 temperature
```


Base code:

```
int32_t current = get_current(raw_i);
int32_t voltage = get_voltage(raw_v);
int32_t temp    = get_temperature(raw_t);

update_soc(current, dt_ms);
int32_t soc = get_soc();

// Thermal management — fan on above 35°C
if (temp > 35) fan_on();
else if (temp < 30) fan_off();

// Motor control — run motor if SoC > 20%
if (soc > 20) motor_on();
else        motor_off();

// Protection checks
check_protection(voltage, current, temp, soc);

// LCD display
// Line 1: V:7.4V I:1.2A
lcd_set_cursor(0, 0);
lcd_print("V:");
lcd_print_number(voltage / 10);
lcd_char('.');
lcd_print_number(voltage % 10);
lcd_print("V I:");
lcd_print_number(current / 10);
lcd_char('.');
lcd_print_number(current % 10);
lcd_print("A ");

// Line 2: T:28C SoC:95%
lcd_set_cursor(1, 0);
lcd_print("T:");
lcd_print_number(temp);
lcd_print("C SoC:");
lcd_print_number(soc);
lcd_print("% ");

// UART CSV log
print_number(voltage);    usart2_send_char(',');
print_number(current);    usart2_send_char(',');
print_number(temp);       usart2_send_char(',');
print_number(soc);        usart2_send_char(',');
```

Base code:

Cube mx code :

```
MX_GPIO_Init();
```

```
MX_ADC1_Init();
```

```
MX_I2C1_Init();
```

```
MX_USART2_UART_Init();
```

```
MX_DMA_Init();
```

```
// Start ADC with DMA
```

```
HAL_ADC_Start_DMA(&hadc1, adc_buf, 3);
```

```
// Read sensor
```

```
float voltage = (adc_buf[0] * 3.3f / 4095.0f) * 6.6f;
```

```
// Send to LCD
```

```
lcd_print("V:7.4V");
```

```
// Send to UART
```

```
HAL_UART_Transmit(&huart2, data, len, 100);
```

```
// Control fan
```

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
```